EN3160 Assignment 3

Tharushika G.K.E.
230636K

GitHub: https://github.com/ErandiTharushika/Assignment-3-Neural-Networks

Setup

In [1]:
```
import copy
import os
import random
import time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils.data import DataLoader, TensorDataset
import torchvision
from torchvision import datasets, transforms, models
from torchvision.datasets.utils import download_and_extract_archive
from IPython.display import display, Markdown

SEED = 636
DATA = Path('data')
RESULTS = Path('results_230636K')
DATA.mkdir(exist_ok=True)
RESULTS.mkdir(exist_ok=True)
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
torch.set_num_threads(min(4, os.cpu_count() or 1))

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = False
    torch.backends.cudnn.deterministic = True

def make_loader(dataset, batch_size, shuffle=False, seed=SEED):
    generator = torch.Generator().manual_seed(seed)
    return DataLoader(dataset, batch_size=batch_size, shuffle=shuffle,
                      num_workers=0, generator=generator,
                      pin_memory=(DEVICE.type == 'cuda'))

set_seed(SEED)
plt.rcParams.update({'figure.dpi': 110, 'font.size': 10,
                     'axes.spines.top': False, 'axes.spines.right': False})
print('PyTorch:', torch.__version__)
print('Torchvision:', torchvision.__version__)
print('Device:', DEVICE)
print('Random seed:', SEED)
```

```
PyTorch: 2.14.1+cpu
Torchvision: 0.29.1+cpu
Device: cpu
Random seed: 636
```

In [2]:
```
@torch.no_grad()
def evaluate(model, loader, predictions=False):
    if isinstance(model, nn.Module):
        model.eval()
    total_loss, correct, count = 0.0, 0, 0
    actual, predicted = [], []
```

```
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        scores = model(images)
        total_loss += F.cross_entropy(scores, labels, reduction='sum').item()
        guesses = scores.argmax(1)
        correct += (guesses == labels).sum().item()
        count += labels.numel()
        if predictions:
            actual.append(labels.cpu())
            predicted.append(guesses.cpu())
    result = {'loss': total_loss / count, 'accuracy': 100 * correct / count}
    if predictions:
        result['actual'] = torch.cat(actual).numpy()
        result['predicted'] = torch.cat(predicted).numpy()
    return result

def save_plot(filename):
    plt.tight_layout()
    plt.savefig(RESULTS / filename, dpi=160, bbox_inches='tight')
    plt.show()
    plt.close()

def learning_plot(history, title, split='test'):
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
    for ax, metric, label in zip(axes, ['loss', 'accuracy'],
                                 ['Cross entropy', 'Accuracy (%)']):
        ax.plot(history['epoch'], history['train_' + metric],
                marker='o', markersize=3, label='Training', color='#267861')
        ax.plot(history['epoch'], history[split + '_' + metric],
                marker='s', markersize=3, label=split.capitalize(), color='#9c4d82')
        ax.set(xlabel='Epoch', ylabel=label, title=title + ': ' + label)
        ax.set_xticks(history['epoch'])
        ax.grid(alpha=0.2)
        ax.legend()
    save_plot(title.lower().replace(' ', '_') + '.png')
```

Question 1

1(a) Adding the hidden layer

I added 100 sigmoid units between the 3072 inputs and ten outputs.

In [3]:
```
cifar_train_raw = datasets.CIFAR10(str(DATA), train=True, download=True)
cifar_test_raw = datasets.CIFAR10(str(DATA), train=False, download=True)

def cifar_tensors(dataset):
    images = torch.from_numpy(dataset.data).permute(0, 3, 1, 2).contiguous()
    images = images.flatten(1).float().div_(127.5).sub_(1.0)
    targets = torch.tensor(dataset.targets, dtype=torch.long)
    return TensorDataset(images, targets)

cifar_train = cifar_tensors(cifar_train_raw)
cifar_test = cifar_tensors(cifar_test_raw)
cifar_loader = make_loader(cifar_train, 50, shuffle=True)
cifar_train_eval = make_loader(cifar_train, 512)
cifar_test_eval = make_loader(cifar_test, 512)
print('Training samples:', len(cifar_train))
print('Test samples:', len(cifar_test))
print('Input features:', cifar_train.tensors[0].shape[1])
```

```
Training samples: 50000
Test samples: 10000
Input features: 3072
```

1(b) Loss and gradients

I calculate cross entropy and update the weights directly, without autograd.

In [4]:
```
class SigmoidClassifier:
    def __init__(self, inputs=3072, hidden=100, outputs=10):
        self.first = torch.empty(inputs, hidden, device=DEVICE)
        self.second = torch.empty(hidden, outputs, device=DEVICE)
```

```
        nn.init.xavier_normal_(self.first)
        nn.init.xavier_normal_(self.second)
        self.hidden_bias = torch.zeros(hidden, device=DEVICE)
        self.output_bias = torch.zeros(outputs, device=DEVICE)

    def __call__(self, images):
        activation = torch.sigmoid(images @ self.first + self.hidden_bias)
        return activation @ self.second + self.output_bias

    @torch.no_grad()
    def update(self, images, targets, rate):
        activation = torch.sigmoid(images @ self.first + self.hidden_bias)
        logits = activation @ self.second + self.output_bias
        log_probs = F.log_softmax(logits, dim=1)
        loss = -log_probs.gather(1, targets[:, None]).mean()
        expected = F.one_hot(targets, num_classes=10).to(logits.dtype)
        output_error = (log_probs.exp() - expected) / targets.numel()
        hidden_error = (output_error @ self.second.T) * activation * (1 - activation)

        grad_first = images.T @ hidden_error
        grad_second = activation.T @ output_error
        grad_hidden_bias = hidden_error.sum(dim=0)
        grad_output_bias = output_error.sum(dim=0)
        self.first -= rate * grad_first
        self.second -= rate * grad_second
        self.hidden_bias -= rate * grad_hidden_bias
        self.output_bias -= rate * grad_output_bias
        return loss.item()

set_seed(SEED)
dense_model = SigmoidClassifier()
dense_scores = dense_model
dense_update = dense_model.update
print('Architecture: 3072 inputs, 100 sigmoid units, 10 output scores')
print('Parameters:', sum(p.numel() for p in vars(dense_model).values()))
```

```
Architecture: 3072 inputs, 100 sigmoid units, 10 output scores
Parameters: 308310
```

1(c) Ten epochs

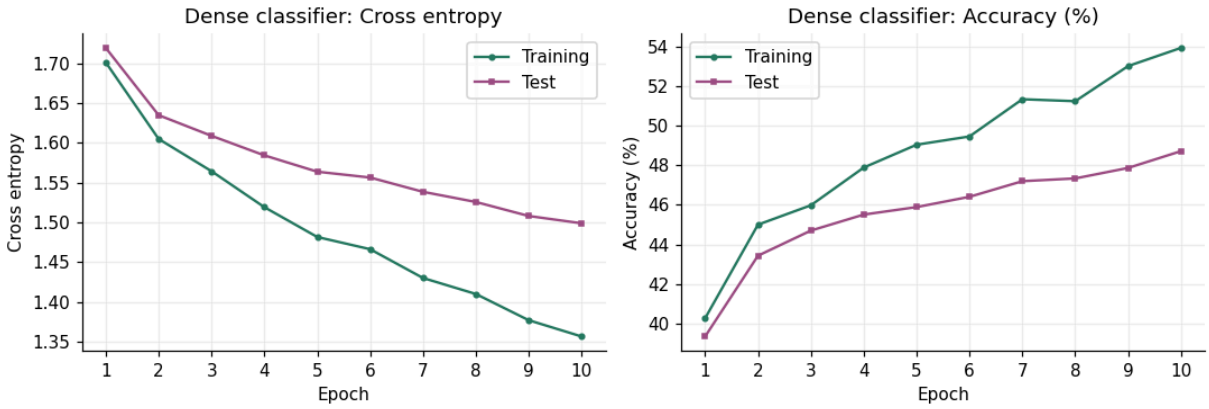
Training starts with Xavier weights and a learning rate of 0.1.

In [5]:
```
dense_rows = []
for epoch in range(1, 11):
    rate = 0.1 * (0.9 ** (epoch - 1))
    for images, labels in cifar_loader:
        dense_update(images.to(DEVICE), labels.to(DEVICE), rate)
    train_metrics = evaluate(dense_scores, cifar_train_eval)
    test_metrics = evaluate(dense_scores, cifar_test_eval)
    dense_rows.append({
        'epoch': epoch, 'learning_rate': rate,
        'train_loss': train_metrics['loss'], 'test_loss': test_metrics['loss'],
        'train_accuracy': train_metrics['accuracy'],
        'test_accuracy': test_metrics['accuracy']
    })
    print(f"Epoch {epoch:2d}/10  Training {train_metrics['accuracy']:.2f}%  "
          f"Test {test_metrics['accuracy']:.2f}%  Test loss {test_metrics['loss']:.4f}")

dense_history = pd.DataFrame(dense_rows)
display(dense_history.round(4))
dense_history.to_csv(RESULTS / 'q1_history.csv', index=False)
learning_plot(dense_history, 'Dense classifier')
```

```
Epoch  1/10  Training 40.26%  Test 39.34%  Test loss 1.7195
Epoch  2/10  Training 45.00%  Test 43.43%  Test loss 1.6349
Epoch  3/10  Training 45.98%  Test 44.70%  Test loss 1.6089
Epoch  4/10  Training 47.89%  Test 45.51%  Test loss 1.5842
Epoch  5/10  Training 49.05%  Test 45.89%  Test loss 1.5637
Epoch  6/10  Training 49.46%  Test 46.41%  Test loss 1.5564
Epoch  7/10  Training 51.35%  Test 47.20%  Test loss 1.5384
Epoch  8/10  Training 51.25%  Test 47.34%  Test loss 1.5256
Epoch  9/10  Training 53.03%  Test 47.87%  Test loss 1.5079
Epoch 10/10  Training 53.95%  Test 48.72%  Test loss 1.4987
```

	epoch	learning_rate	train_loss	test_loss	train_accuracy	test_accuracy
0	1	0.1000	1.7013	1.7195	40.264	39.34
1	2	0.0900	1.6049	1.6349	45.000	43.43
2	3	0.0810	1.5643	1.6089	45.980	44.70
3	4	0.0729	1.5190	1.5842	47.890	45.51
4	5	0.0656	1.4815	1.5637	49.046	45.89
5	6	0.0590	1.4661	1.5564	49.464	46.41
6	7	0.0531	1.4299	1.5384	51.348	47.20
7	8	0.0478	1.4098	1.5256	51.248	47.34
8	9	0.0430	1.3767	1.5079	53.026	47.87
9	10	0.0387	1.3563	1.4987	53.946	48.72



Test accuracy was **48.72%**, below the **53.95%** training score. This gap suggests mild overfitting.

Question 2

Both models use the same padded MNIST images and training settings: Adam, learning rate 0.001, batch size 128 and ten epochs. Reported scores use evaluation mode.

In [7]:
```
mnist_train_raw = datasets.MNIST(str(DATA), train=True, download=True)
mnist_test_raw = datasets.MNIST(str(DATA), train=False, download=True)

def mnist_tensors(dataset):
    images = dataset.data.unsqueeze(1).float().div(255)
    images = F.pad(images, (2, 2, 2, 2))
    images = (images - 0.1307) / 0.3081
    return TensorDataset(images, dataset.targets.clone())

mnist_train = mnist_tensors(mnist_train_raw)
mnist_test = mnist_tensors(mnist_test_raw)
mnist_train_eval = make_loader(mnist_train, 512)
mnist_test_eval = make_loader(mnist_test, 512)
print('Training samples:', len(mnist_train))
print('Test samples:', len(mnist_test))
print('Image shape:', tuple(mnist_train.tensors[0].shape[1:]))
```

```
Training samples: 60000
Test samples: 10000
Image shape: (1, 32, 32)
```

2(a) LeNet5

The 120 unit dense layer does the job of LeNet5's final 5 by 5 convolution.

In [8]:
```
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        self.spatial_layers = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5), nn.Tanh(), nn.AvgPool2d(2),
            nn.Conv2d(6, 16, kernel_size=5), nn.Tanh(), nn.AvgPool2d(2)
        )
        self.decision_layers = nn.Sequential(
            nn.Flatten(), nn.Linear(16 * 5 * 5, 120), nn.Tanh(),
            nn.Linear(120, 84), nn.Tanh(), nn.Linear(84, 10)
        )

    def forward(self, images):
        return self.decision_layers(self.spatial_layers(images))

print(LeNet5())
```

```
LeNet5(
  (spatial_layers): Sequential(
    (0): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))
    (1): Tanh()
    (2): AvgPool2d(kernel_size=2, stride=2, padding=0)
    (3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
    (4): Tanh()
    (5): AvgPool2d(kernel_size=2, stride=2, padding=0)
  )
  (decision_layers): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=400, out_features=120, bias=True)
    (2): Tanh()
    (3): Linear(in_features=120, out_features=84, bias=True)
    (4): Tanh()
    (5): Linear(in_features=84, out_features=10, bias=True)
  )
)
```

In [9]:
```
def train_epoch(model, loader, optimizer, frozen_backbone=False):
    if frozen_backbone:
        # Keep both pretrained weights and BatchNorm statistics fixed.
        model.eval()
        model.fc.train()
    else:
        model.train()
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad(set_to_none=True)
        loss = F.cross_entropy(model(images), labels)
        loss.backward()
        optimizer.step()

def fit_digits(constructor, name):
    set_seed(SEED)
    model = constructor().to(DEVICE)
    training = make_loader(mnist_train, 128, shuffle=True)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    rows = []
    started = time.perf_counter()
    for epoch in range(1, 11):
        train_epoch(model, training, optimizer)
        train = evaluate(model, mnist_train_eval)
        test = evaluate(model, mnist_test_eval)
        rows.append({'epoch': epoch,
                     'train_loss': train['loss'], 'test_loss': test['loss'],
                     'train_accuracy': train['accuracy'],
                     'test_accuracy': test['accuracy']})
        print(f"{name}  {epoch:2d}/10  "
              f"Train loss {train['loss']:.4f}  Test loss {test['loss']:.4f}  "
              f"Train {train['accuracy']:.2f}%  Test {test['accuracy']:.2f}%")
    history = pd.DataFrame(rows)
    history.to_csv(RESULTS / (name.lower() + '_history.csv'), index=False)
    torch.save(model.state_dict(), RESULTS / (name.lower() + '.pt'))
    return model, history, time.perf_counter() - started
```

In [10]:
```
lenet_model, lenet_history, lenet_seconds = fit_digits(LeNet5, 'LeNet5')
```

```
LeNet5   1/10  Train loss 0.1201  Test loss 0.1190  Train 96.56%  Test 96.43%
LeNet5   2/10  Train loss 0.0749  Test loss 0.0796  Train 97.70%  Test 97.38%
LeNet5   3/10  Train loss 0.0499  Test loss 0.0587  Train 98.49%  Test 98.15%
LeNet5   4/10  Train loss 0.0410  Test loss 0.0519  Train 98.73%  Test 98.31%
LeNet5   5/10  Train loss 0.0282  Test loss 0.0437  Train 99.13%  Test 98.70%
LeNet5   6/10  Train loss 0.0233  Test loss 0.0467  Train 99.27%  Test 98.52%
LeNet5   7/10  Train loss 0.0203  Test loss 0.0441  Train 99.34%  Test 98.55%
LeNet5   8/10  Train loss 0.0160  Test loss 0.0413  Train 99.53%  Test 98.67%
LeNet5   9/10  Train loss 0.0197  Test loss 0.0472  Train 99.37%  Test 98.47%
LeNet5  10/10  Train loss 0.0145  Test loss 0.0442  Train 99.55%  Test 98.66%
```

2(b) Small CNN

Batch normalization and dropout are added to the three convolution blocks.

In [11]:
```python
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        blocks = []
        for inputs, outputs in [(1, 16), (16, 32), (32, 64)]:
            blocks.extend([
                nn.Conv2d(inputs, outputs, 3, padding=1, bias=False),
                nn.BatchNorm2d(outputs), nn.ReLU(), nn.MaxPool2d(2)
            ])
        self.features = nn.Sequential(*blocks)
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(64 * 4 * 4, 64), nn.ReLU(),
            nn.Dropout(p=0.4), nn.Linear(64, 10)
        )

    def forward(self, images):
        return self.classifier(self.features(images))

print(SmallCNN())
```

```
SmallCNN(
  (features): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (5): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (9): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (10): ReLU()
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1024, out_features=64, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.4, inplace=False)
    (4): Linear(in_features=64, out_features=10, bias=True)
  )
)
```

In [12]:
```python
cnn_model, cnn_history, cnn_seconds = fit_digits(SmallCNN, 'Small_CNN')
```

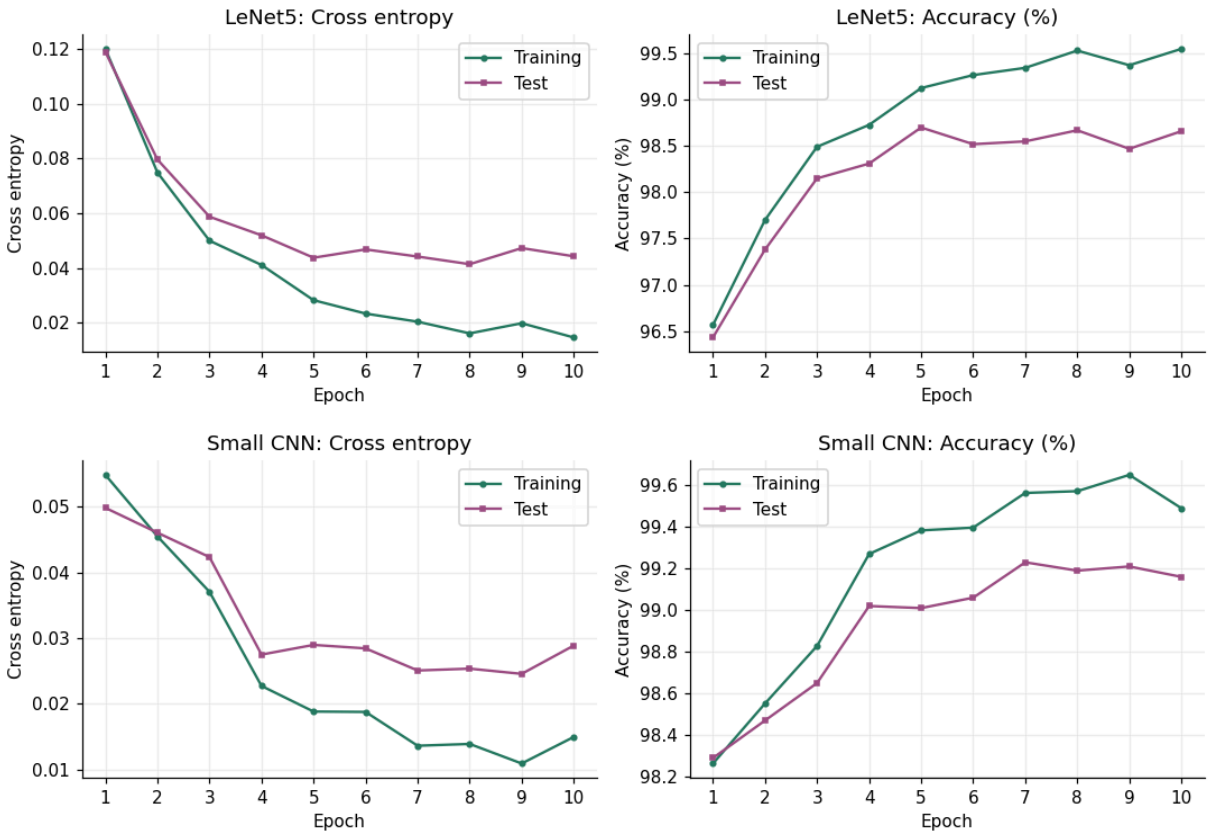
```
Small_CNN   1/10  Train loss 0.0548  Test loss 0.0498  Train 98.26%  Test 98.29%
Small_CNN   2/10  Train loss 0.0455  Test loss 0.0461  Train 98.55%  Test 98.47%
Small_CNN   3/10  Train loss 0.0371  Test loss 0.0423  Train 98.83%  Test 98.65%
Small_CNN   4/10  Train loss 0.0227  Test loss 0.0275  Train 99.27%  Test 99.02%
Small_CNN   5/10  Train loss 0.0188  Test loss 0.0290  Train 99.38%  Test 99.01%
Small_CNN   6/10  Train loss 0.0188  Test loss 0.0285  Train 99.40%  Test 99.06%
Small_CNN   7/10  Train loss 0.0136  Test loss 0.0251  Train 99.56%  Test 99.23%
Small_CNN   8/10  Train loss 0.0139  Test loss 0.0254  Train 99.57%  Test 99.19%
Small_CNN   9/10  Train loss 0.0109  Test loss 0.0246  Train 99.65%  Test 99.21%
Small_CNN  10/10  Train loss 0.0150  Test loss 0.0289  Train 99.49%  Test 99.16%
```

2(c) Comparison

Actual digits are on rows; predictions are on columns.

In [13]:
```
mnist_runs = [('LeNet5', lenet_model, lenet_history, lenet_seconds),
              ('Small CNN', cnn_model, cnn_history, cnn_seconds)]
comparison_rows = []
for name, model, history, seconds in mnist_runs:
    last = history.iloc[-1]
    comparison_rows.append({
        'Model': name,
        'Parameters': sum(p.numel() for p in model.parameters()),
        'Train accuracy (%)': last.train_accuracy,
        'Test accuracy (%)': last.test_accuracy,
        'Train loss': last.train_loss, 'Test loss': last.test_loss,
        'Time (s)': seconds
    })
mnist_results = pd.DataFrame(comparison_rows).set_index('Model')
display(mnist_results.round(4))
mnist_results.to_csv(RESULTS / 'q2_comparison.csv')
learning_plot(lenet_history, 'LeNet5')
learning_plot(cnn_history, 'Small CNN')
```

Model	Parameters	Train accuracy (%)	Test accuracy (%)	Train loss	Test loss	Time (s)
LeNet5	61706	99.55	98.66	0.0145	0.0442	72.5308
Small CNN	89658	99.49	99.16	0.0150	0.0289	208.1097

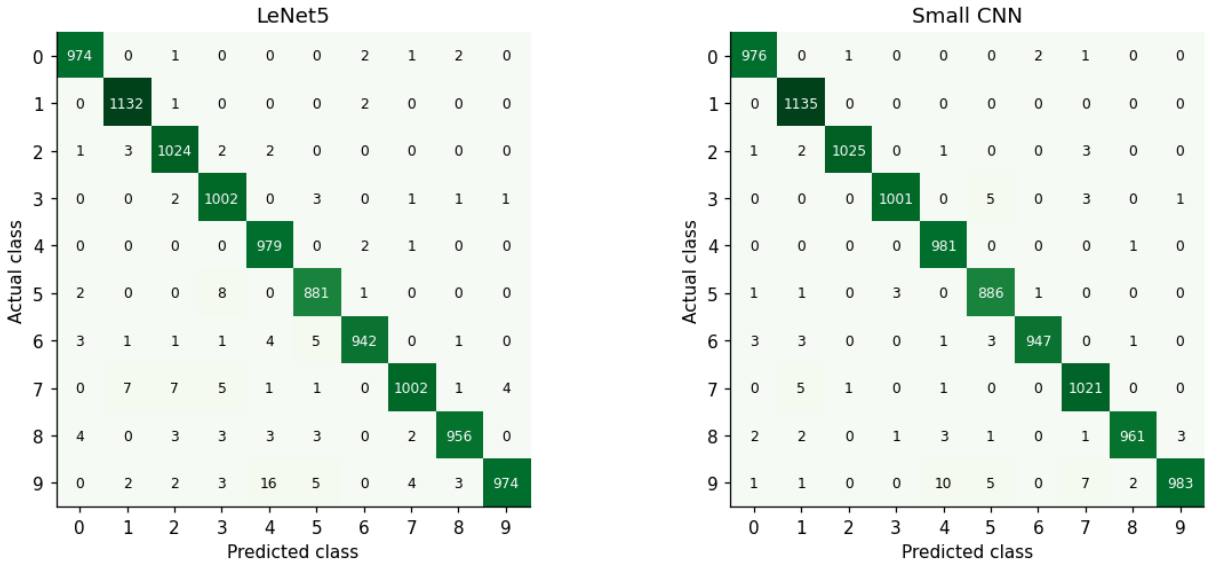


In [14]:
```
def confusion_table(actual, predicted, classes):
    counts = np.zeros((classes, classes), dtype=int)
    np.add.at(counts, (actual, predicted), 1)
    return counts

def plot_confusion(ax, matrix, names, title):
    ax.imshow(matrix, cmap='Greens')
    ax.set(xticks=range(len(names)), yticks=range(len(names)),
           xticklabels=names, yticklabels=names,
           xlabel='Predicted class', ylabel='Actual class', title=title)
    threshold = matrix.max() / 2
    for row in range(len(names)):
        for column in range(len(names)):
            ax.text(column, row, str(matrix[row, column]), ha='center', va='center',
```

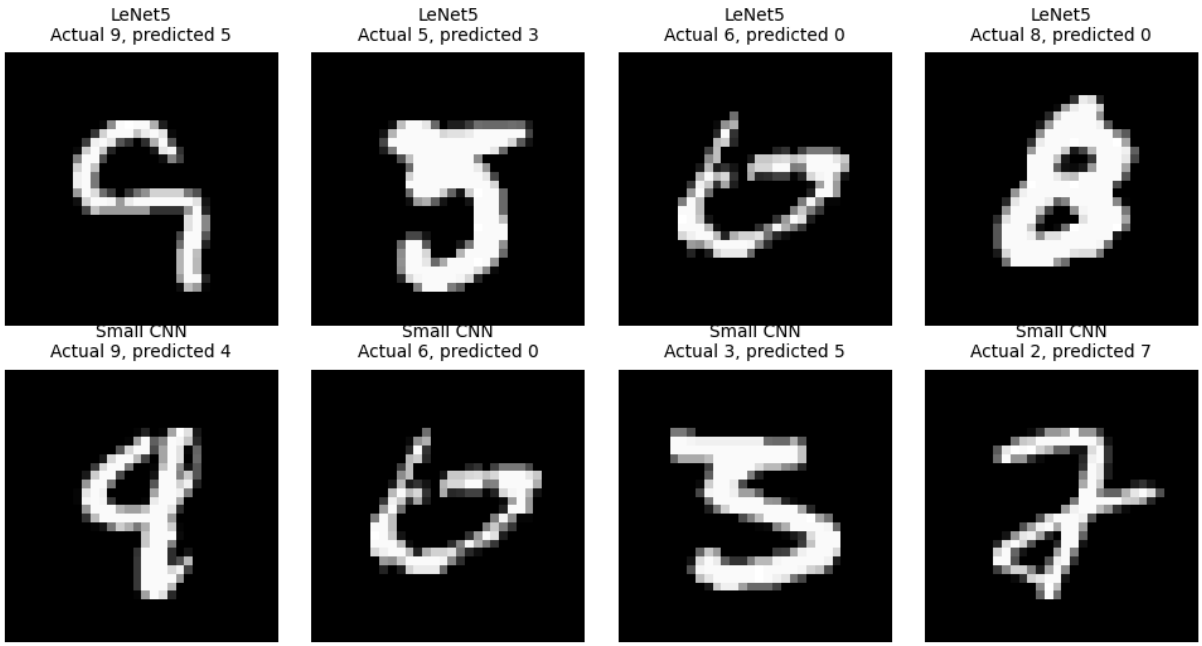
```
                    fontsize=8, color='white' if matrix[row, column] > threshold else 'black')

digit_predictions, digit_matrices = {}, {}
fig, axes = plt.subplots(1, 2, figsize=(11, 4.7))
for ax, (name, model, _, _) in zip(axes, mnist_runs):
    result = evaluate(model, mnist_test_eval, predictions=True)
    matrix = confusion_table(result['actual'], result['predicted'], 10)
    assert matrix.sum() == len(mnist_test)
    digit_predictions[name] = result
    digit_matrices[name] = matrix
    plot_confusion(ax, matrix, list(range(10)), name)
    pd.DataFrame(matrix).to_csv(RESULTS / (name.lower().replace(' ', '_') + '_confusion.csv'))
save_plot('q2_confusion.png')
```



In [15]:

```
fig, axes = plt.subplots(2, 4, figsize=(9, 4.8))
for row, (name, result) in enumerate(digit_predictions.items()):
    mistakes = np.flatnonzero(result['actual'] != result['predicted'])[:4]
    for ax in axes[row]:
        ax.axis('off')
    for ax, index in zip(axes[row], mistakes):
        image = mnist_test.tensors[0][index, 0] * 0.3081 + 0.1307
        ax.imshow(image, cmap='gray', vmin=0, vmax=1)
        ax.set_title(f"{name}\nActual {result['actual'][index]}, predicted {result['predicted'][index]}"
                     fontsize=9)
save_plot('q2_mistakes.png')
```



The small CNN made 84 test errors, compared with 134 for LeNet5. Both struggled most with 9 being read as 4.

Question 3

Both runs start from the same ImageNet1K weights and new classifier. I compare them over ten epochs using the same images, augmentation and SGD settings.

In [17]:
```
insect_root = DATA / 'hymenoptera_data'
if not all((insect_root / split / category).is_dir()
           for split in ['train', 'val'] for category in ['ants', 'bees']):
    download_and_extract_archive(
        'https://download.pytorch.org/tutorial/hymenoptera_data.zip',
        download_root=str(DATA), extract_root=str(DATA)
    )

image_mean = [0.485, 0.456, 0.406]
image_std = [0.229, 0.224, 0.225]
training_transform = transforms.Compose([
    transforms.RandomResizedCrop(224), transforms.RandomHorizontalFlip(),
    transforms.ToTensor(), transforms.Normalize(image_mean, image_std)
])
evaluation_transform = transforms.Compose([
    transforms.Resize(256), transforms.CenterCrop(224),
    transforms.ToTensor(), transforms.Normalize(image_mean, image_std)
])
insect_train = datasets.ImageFolder(str(insect_root / 'train'), training_transform)
insect_train_clean = datasets.ImageFolder(str(insect_root / 'train'), evaluation_transform)
insect_validation = datasets.ImageFolder(str(insect_root / 'val'), evaluation_transform)
class_names = insect_train.classes
assert insect_validation.class_to_idx == insect_train.class_to_idx
insect_train_eval = make_loader(insect_train_clean, 32)
insect_val_eval = make_loader(insect_validation, 32)
print('Classes:', class_names)
print('Training images:', len(insect_train))
print('Validation images:', len(insect_validation))
```

```
Classes: ['ants', 'bees']
Training images: 244
Validation images: 153
```

In [18]:
```
def make_resnet(freeze_features):
    set_seed(SEED)
    network = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
    if freeze_features:
        network.requires_grad_(False)
    # The newly created classification layer remains trainable.
    network.fc = nn.Linear(512, len(class_names))
    return network.to(DEVICE)
```

In [19]:
```
TRANSFER_EPOCHS = 10

def fit_transfer(freeze_features, label):
    model = make_resnet(freeze_features)
    trainable = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.SGD(trainable, lr=0.001, momentum=0.9)
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)
    loader = make_loader(insect_train, 4, shuffle=True)
    history_rows = []
    best_accuracy, best_epoch, best_weights = -1.0, 0, None
    frozen_start = None
    if freeze_features:
        frozen_start = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()
                        if not k.startswith('fc.')}
    started = time.perf_counter()
    for epoch in range(1, TRANSFER_EPOCHS + 1):
        rate = optimizer.param_groups[0]['lr']
        train_epoch(model, loader, optimizer, frozen_backbone=freeze_features)
        train = evaluate(model, insect_train_eval)
        validation = evaluate(model, insect_val_eval)
        history_rows.append({
            'epoch': epoch, 'learning_rate': rate,
            'train_loss': train['loss'], 'val_loss': validation['loss'],
            'train_accuracy': train['accuracy'], 'val_accuracy': validation['accuracy']
        })
```

```
        if validation['accuracy'] > best_accuracy:
            best_accuracy, best_epoch = validation['accuracy'], epoch
            best_weights = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
        scheduler.step()
        print(f"{label}  {epoch:2d}/{TRANSFER_EPOCHS}  "
              f"Train {train['accuracy']:.2f}%  Validation {validation['accuracy']:.2f}%  "
              f"Validation loss {validation['loss']:.4f}")
    if freeze_features:
        assert all(torch.equal(before, model.state_dict()[key].cpu())
                   for key, before in frozen_start.items())
        print('Pretrained weights and batch statistics remained fixed.')
    seconds = time.perf_counter() - started
    history = pd.DataFrame(history_rows)
    stem = label.lower().replace(' ', '_')
    history.to_csv(RESULTS / (stem + '_history.csv'), index=False)
    model.load_state_dict(best_weights)
    torch.save({'state_dict': best_weights, 'classes': class_names, 'epoch': best_epoch},
               RESULTS / (stem + '_best.pt'))
    best_result = evaluate(model, insect_val_eval, predictions=True)
    summary = {
        'Method': label, 'Trainable parameters': sum(p.numel() for p in trainable),
        'Final train (%)': history.iloc[-1].train_accuracy,
        'Final validation (%)': history.iloc[-1].val_accuracy,
        'Best validation (%)': best_result['accuracy'],
        'Best epoch': best_epoch, 'Time (s)': seconds
    }
    return model, history, best_result, summary
```

3(a) Fine tuning

Both the features and classifier are trainable.

In [20]:
```
fine_model, fine_history, fine_result, fine_summary = fit_transfer(False, 'Fine tuning')
```

```
Fine tuning   1/10  Train 96.31%  Validation 94.77%  Validation loss 0.1553
Fine tuning   2/10  Train 97.54%  Validation 92.16%  Validation loss 0.1846
Fine tuning   3/10  Train 97.54%  Validation 90.85%  Validation loss 0.2554
Fine tuning   4/10  Train 93.44%  Validation 88.24%  Validation loss 0.3362
Fine tuning   5/10  Train 96.72%  Validation 91.50%  Validation loss 0.3360
Fine tuning   6/10  Train 93.44%  Validation 87.58%  Validation loss 0.2965
Fine tuning   7/10  Train 97.13%  Validation 86.27%  Validation loss 0.2711
Fine tuning   8/10  Train 98.77%  Validation 87.58%  Validation loss 0.2573
Fine tuning   9/10  Train 99.18%  Validation 92.81%  Validation loss 0.1926
Fine tuning  10/10  Train 99.18%  Validation 92.16%  Validation loss 0.2106
```

3(b) Fixed features

The backbone is fixed, including its batch normalization statistics. Only the classifier is updated.

In [21]:
```
fixed_model, fixed_history, fixed_result, fixed_summary = fit_transfer(True, 'Feature extraction')
```

```
Feature extraction   1/10  Train 97.13%  Validation 94.77%  Validation loss 0.1526
Feature extraction   2/10  Train 97.13%  Validation 95.42%  Validation loss 0.1508
Feature extraction   3/10  Train 97.95%  Validation 96.08%  Validation loss 0.1395
Feature extraction   4/10  Train 97.95%  Validation 95.42%  Validation loss 0.1480
Feature extraction   5/10  Train 98.36%  Validation 94.12%  Validation loss 0.1880
Feature extraction   6/10  Train 97.54%  Validation 95.42%  Validation loss 0.1488
Feature extraction   7/10  Train 98.77%  Validation 95.42%  Validation loss 0.1389
Feature extraction   8/10  Train 98.77%  Validation 96.08%  Validation loss 0.1380
Feature extraction   9/10  Train 99.18%  Validation 96.08%  Validation loss 0.1384
Feature extraction  10/10  Train 99.18%  Validation 96.08%  Validation loss 0.1418
Pretrained weights and batch statistics remained fixed.
```

Results

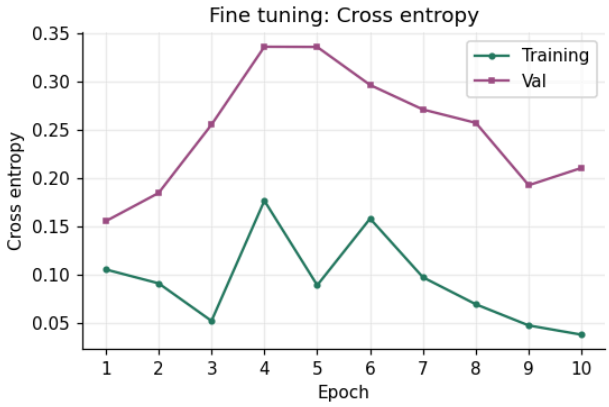
Curves show all ten epochs. Prediction examples and confusion matrices use the epoch with the highest validation accuracy.

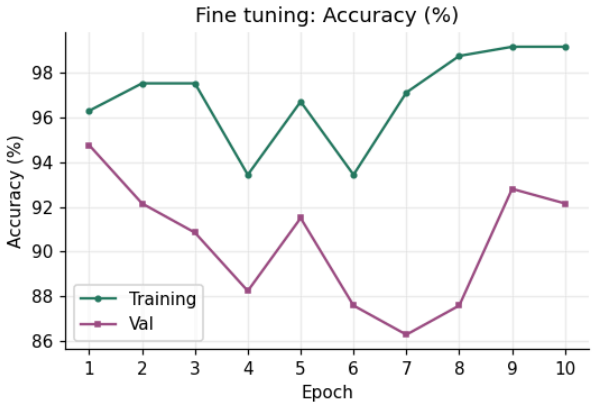
In [22]:
```
transfer_results = pd.DataFrame([fine_summary, fixed_summary]).set_index('Method')
display(transfer_results.round(3))
transfer_results.to_csv(RESULTS / 'q3_comparison.csv')
learning_plot(fine_history, 'Fine tuning', split='val')
```

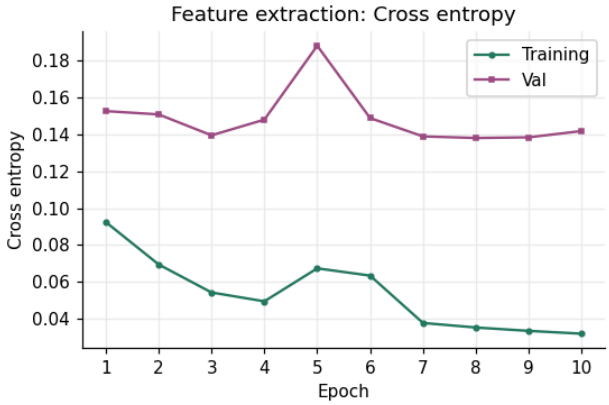
```
learning_plot(fixed_history, 'Feature extraction', split='val')

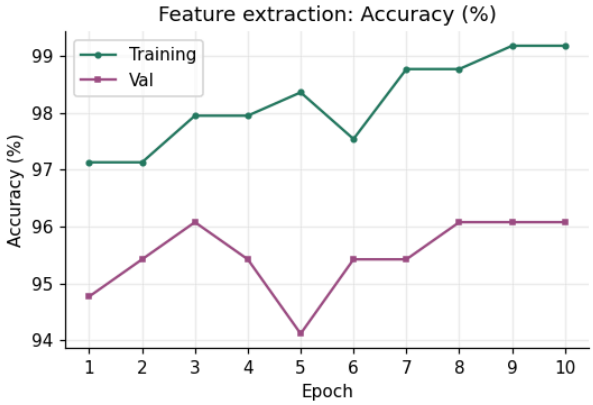
fig, axes = plt.subplots(1, 2, figsize=(8.5, 3.5))
for ax, result, label in zip(axes, [fine_result, fixed_result],
                             ['Fine tuning', 'Feature extraction']):
    matrix = confusion_table(result['actual'], result['predicted'], len(class_names))
    assert matrix.sum() == len(insect_validation)
    plot_confusion(ax, matrix, class_names, label)
save_plot('q3_confusion.png')
```

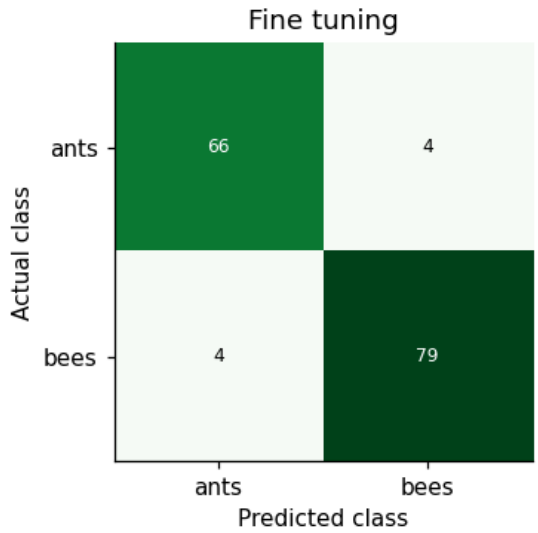
Method	Trainable parameters	Final train (%)	Final validation (%)	Best validation (%)	Best epoch	Time (s)
Fine tuning	11177538	99.18	92.157	94.771	1	245.602
Feature extraction	1026	99.18	96.078	96.078	3	146.970

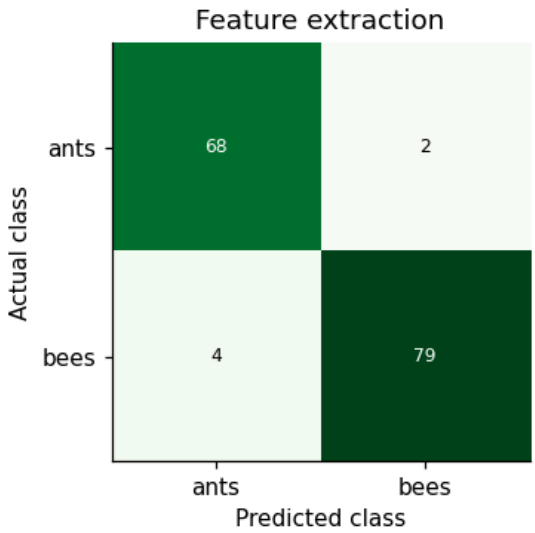






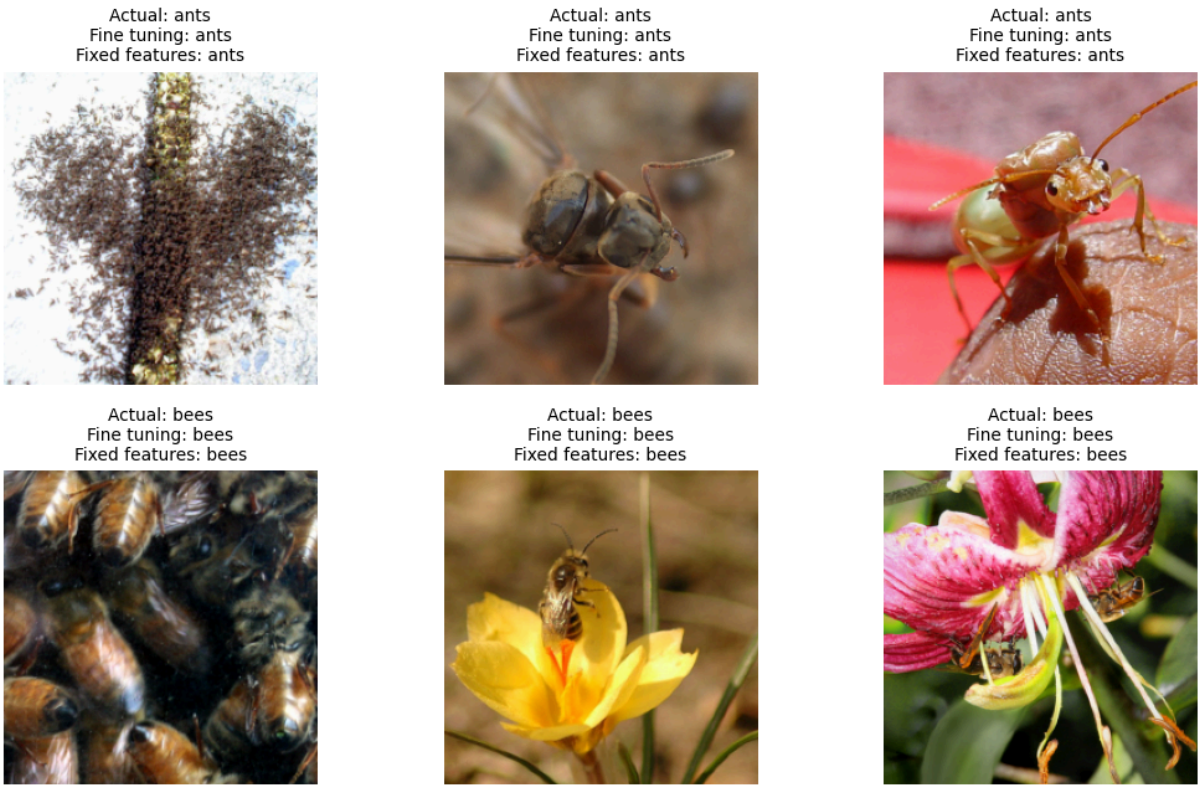






In [23]:
```
targets = np.array(insect_validation.targets)
sample_indices = np.concatenate([np.flatnonzero(targets == label)[:3]
                                 for label in range(len(class_names))])
```

```
fig, axes = plt.subplots(2, 3, figsize=(10, 6))
for ax, index in zip(axes.flat, sample_indices):
    tensor, label = insect_validation[int(index)]
    rgb = tensor.permute(1, 2, 0).numpy()
    rgb = np.clip(rgb * image_std + image_mean, 0, 1)
    ax.imshow(rgb)
    ft_name = class_names[fine_result['predicted'][index]]
    fx_name = class_names[fixed_result['predicted'][index]]
    ax.set_title(f'Actual: {class_names[label]}\nFine tuning: {ft_name}\nFixed features: {fx_name}',
                 fontsize=9)
    ax.axis('off')
save_plot('q3_predictions.png')
```



Feature extraction reached **96.08%**, while fine tuning reached **94.77%**. Training only the classifier worked better in this run. The best epoch was chosen using this validation set, so a separate test would be needed to confirm the comparison.